

A Hierarchical Framework With Spatio-Temporal Consistency Learning for Emergence Detection in Complex Adaptive Systems

Siyuan Chen¹, Xin Du¹, and Jiahai Wang¹, *Senior Member, IEEE*

Abstract—Emergence, a global property of complex adaptive systems (CASs) constituted by interactive agents, is prevalent in real-world dynamic systems, e.g., network-level traffic congestions. Detecting its formation and evaporation helps to monitor the state of a system, allowing it to issue a warning signal for harmful emergent phenomena. Since there is no centralized controller of CAS, detecting emergence based on each agent's local observation is desirable but challenging. Existing works are unable to capture emergence-related spatial patterns, and fail to model the nonlinear relationships among agents. This article proposes a hierarchical framework with spatio-temporal consistency learning (HSTCL) to solve these two problems by learning the system representation and agent representations, respectively. Spatio-temporal encoders (STEs) composed of spatial and temporal transformers are designed to capture agents' nonlinear relationships and the system's complex evolution. Agents' and the system's representations are learned to preserve the spatio-temporal consistency by minimizing the spatial and temporal dissimilarities in a self-supervised manner in the latent space. Our method achieves more accurate detection than traditional methods and deep learning methods on three datasets with well-known yet hard-to-detect emergent behaviors. Notably, our hierarchical framework is generic in incorporating other deep learning methods for agent-level and system-level detection.

Index Terms—Complex adaptive systems (CASs), emergence detection, self-supervised learning on dynamic graphs, spatio-temporal modeling.

I. INTRODUCTION

MANY real-world dynamic systems can be regarded as complex adaptive systems (CASs) composed of

Received 17 January 2024; revised 20 July 2024 and 19 September 2024; accepted 28 September 2024. Date of publication 21 October 2024; date of current version 9 July 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2018AAA0101203; in part by the National Natural Science Foundation of China under Grant 62406083, Grant 62472461, and Grant 62072483; and in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2022A1515011690. (Corresponding author: Jiahai Wang.)

Siyuan Chen is with the School of Computer Science and Cyber Engineering, Guangzhou University, Guangzhou 510006, China (e-mail: chensiyuan@gzhu.edu.cn).

Xin Du is with the Civil Aviation Electronic Information Engineering College, Guangzhou Civil Aviation College, Guangzhou 510403, China (e-mail: duxin@gcac.edu.cn).

Jiahai Wang is with the School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou 510275, China (e-mail: wangjiahai@mail.sysu.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TNNLS.2024.3477320>, provided by the authors.

Digital Object Identifier 10.1109/TNNLS.2024.3477320

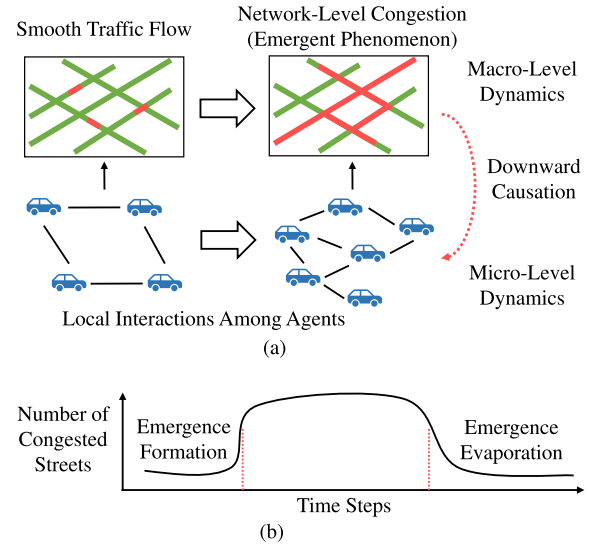


Fig. 1. (a) Illustrates the emergence through the traffic flow. (b) Shows that emergence detection is framed as a CPD problem.

autonomous, adaptive, and interacting agents, whose interactions at the micro level can result in emergent phenomena at the macro level, namely, emergence [1], [2], [3]. Fig. 1(a) presents an example. When there is adequate space among cars, the road net enjoys a smooth traffic flow. When the distances between cars significantly narrow down, network-level congestion occurs as an emergent phenomenon. Emergence is irreducible to the properties of agents that constitute CAS, and it is unpredictable by nature [1], [4]. Nonetheless, it will be beneficial to detect the formation and evaporation of emergence, specifically, weak emergence that is scientifically relevant [5]. It can help monitor some global properties of the system and issue a warning signal when an undesirable phenomenon arrives. For example, reporting a traffic jam based on the feedback of cars can help with the health management of traffic systems. It will complement existing monitors relying on static devices like sensors or cameras [6].

Emergence detection can be formulated as an online change-point detection (CPD) problem by regarding the time steps when emergence forms or evaporates as change points [7], [8]. As shown in Fig. 1(b), the number of congested streets can serve as a global variable to monitor the emergence. However, CASs are distributed by nature, i.e., there is no

centralized controller that can access the states of all agents. Therefore, methods requiring all agents' states to compute a global metric for emergence detection become impractical under the distributed setting. Hence, it is desirable to detect emergence using agents' local observation, which is feasible because all agents experience the downward causation [7] when the emergence forms, as shown in Fig. 1(a). For example, cars slow down in a crowded street. Based on this observation, O'toole et al. [7] proposed DETect, the only feasible emergence detection method for the distributed setting. In DETect, each agent analyzes its relationship with neighbors, communicates with them, and sends feedback when finding a noticeable change in the relationship. DETect concludes the formation or evaporation of emergence when the number of feedback gets significantly larger.

Though making a big step, DETect has two main limitations. First, it simply counts agents' feedback to monitor the emergence, which may neglect the spatial patterns that are highly correlated to emergence. Second, it adopts linear regression to model an agent's relationship with its neighbors, which may fail to capture nonlinear relationships.

This article tries to overcome the above limitations from a graph perspective. CAS can be regarded as a dynamic graph [9], and thus emergence detection can be cast to online CPD in dynamic graphs under the distributed setting. Based on this formulation, it suffices to learn a system-level representation aware of emergence-related patterns, and agent-level representations encoding the nonlinear relationships. When emergence forms or evaporates, the system representations between adjacent time steps are expected to be inconsistent, which can serve as a detecting signal for emergence. Specifically, a hierarchical framework with spatio-temporal consistency learning (HSTCL) is proposed for emergence detection. HSTCL is of a three-layer structure, "agents-region monitors-global monitor," which allows to capturing of emergence-related spatial patterns by aggregating agent-level detecting results from bottom-up. HSTCL can be conceptually implemented by the efficient end-edge-cloud collaborative framework. Spatio-temporal encoders (STEs) composed of spatial and temporal transformers are designed to model the complex variation of agents' nonlinear relationships and the system states in highly dynamic scenarios. Representations of agents and the system are learned to jointly preserve the spatial and temporal consistency by, respectively, minimizing the spatial and temporal dissimilarities in the latent space. Compared with DETect, HSTCL can capture nonlinear spatio-temporal relationships with the aid of STEs, and identify system-wide emergence-related spatial patterns beyond agents' scope. As a framework, HSTCL is more flexible than DETect. It can incorporate other deep learning methods to further boost the performance. Our contributions are summarized as follows.

- 1) The hierarchical framework HSTCL can capture emergence-related spatial patterns by aggregating agent-level detecting results from the bottom-up.
- 2) STEs composed of spatial and temporal transformers are designed to model the nonlinear relationships among agents and the evolution of the system. Featured by parallel execution and incremental updates of

representations, these encoders are especially suitable for online detection.

- 3) The agent representations and the system representation are learned to preserve the intrinsic spatio-temporal consistency in a self-supervised manner. The training strategy avoids data augmentations that may break spatio-temporal consistency and negative samples that increase the computational overhead.
- 4) Extensive experiments on three datasets with well-known yet hard-to-detect emergent phenomena validate the superiority of HSTCL over DETect and deep learning methods. Notably, other deep learning methods can be incorporated into our framework for emergence detection.

II. RELATED WORK

A. Emergence Detection

Detecting the emergence of CAS, generally requires at least one global monitor [2]. Depending on how the monitor acquires the information of agents for detection, existing methods fall into three design choices of architectures: 1) a single monitor with direct access to agents' states; 2) a monitor collecting agents' information indirectly, e.g., from static sensors; and 3) a monitor collecting feedback from mobile agents. Our method belongs to class 3.

Class 1 methods define global variables to monitor the system state, e.g., information entropy [10], [11], [12], [13], statistical divergence [14], and Granger-emergence [15]. These methods require centralized monitoring, and are thus inapplicable under the distributed setting. Class 2 methods allow distributed monitoring. However, they require prior knowledge of emergence to decide what to detect at each sensor [16], [17], falling short in detecting unknown emergent phenomena. DETect [7], the only existing method from class 3, overcomes the limitations of the first two classes. Each agent serves as a local detector, and agents' feedback is aggregated to monitor the emergence. Our method inherits the advantages of DETect, and further introduces region monitors between agents and the global monitor, allowing us to analyze spatial patterns ignored by DETect. STEs are tailored to model nonlinear relationships among agents, which is difficult for the linear models adopted by DETect.

Similar to region monitoring, Santos and Zhao [18] detect emergence by utilizing subsystem-level information. Their work requires collecting and labeling data of predefined subsystems, which is not applicable to emergence detection rooted in agents' local observations. More backgrounds of CAS and emergence, along with a detailed description of DETect are shown in Appendix A (supplementary material).

B. Related Graph Mining Tasks

Emergence detection can be viewed as CPD in dynamic graphs [19], [20]. It is also closely related to graph-level anomaly detection (AD) [21], [22] and multivariate time series AD [23], [24], since emergence is a novel global property. Structural changes from the perspectives of edges [25], single-view [26] and multiview [19] graph Laplacian have been

sustainably explored for CPD. Finer-grained detection is also studied on overlapping communities with respect to different stages of evolution [27].

Nonetheless, these methods cannot jointly model the changes in node features and structures. Graph neural networks (GNNs) [28] can overcome this limitation. sGNN [20] adopts siamese GNNs to compare the similarity between two adjacent graph snapshots, but it ignores the temporal dependence of graph snapshots. An offline detection method [29] uses the Gaussian mixture model to cluster the graph snapshots and identifies a change point when adjacent graph snapshots belong to different clusters. However, it is unsuitable for online detection because it needs to acquire the information of graphs over all time steps.

For graph-level AD, the structural changes of dynamic networks are studied from the levels of nodes, communities, and the full-graph [30], [31]. Related multiscale dependence is also explored via graph framelets [32] and graph contrastive learning [33] in general graph learning methods. For time series AD, the anomaly-related multiscale spatio-temporal patterns are modeled by dilated temporal convolution and multihop GNNs [24], and the cross-time spatial dependence is modeled by the fuzzy embedding [34]. The anomaly is measured by prediction error [23], [24], one-class classification loss [35], and contrastive loss [36]. However, these methods are originally designed for centralized detection. Protogerou et al. [37] proposed a distributed graph AD method, where each node shares the latent vector with its neighbors. This will raise privacy concerns and increase the communication cost. Thus, methods from graph-level CPD and AD are inapplicable for emergence detection, but they can adapt to our framework regarding the distributed settings [7], where agents can only sense their neighbors' states and share the detecting results.

C. Self-Supervised Learning for Spatio-Temporal Data

Rich deep learning methods that capture multiscale spatio-temporal correlations have been developed for spatio-temporal data like videos [38] and dynamic graphs [39], with applications to spatio-temporal forecasting [39], task scheduling [40], and decision-making [41]. However, the scarcity of labels makes it difficult to effectively train complex models. Self-supervised learning is explored to leverage rich information underlying the unlabeled data, with success in time series [42], videos [43], and static graphs [33], [44]. However, the efforts in dynamic graphs are limited. Contrastive learning is a typical paradigm, but it is nontrivial to construct different views of a node or a graph that preserve spatio-temporal semantics [45], [46]. Besides, the large number of negative samples substantially increases the computational overhead. To avoid these issues, this article develops noncontrastive spatio-temporal learning strategies.

III. PROPOSED METHOD

A. Problem Formulation

Regarding the time steps when the emergence forms or evaporates as change points, emergence detection in CAS can be formulated as CPD in dynamic graphs as follows.

Definition 1 (Dynamic Graph): A dynamic graph is composed of a graph series $\{\mathcal{G}'\}_{t=1}^T$, where $\mathcal{G}' = (\mathcal{V}, \mathcal{E}', \mathbf{X}')$ is a snapshot at time t , with $\mathcal{V}$ as the set of nodes over all time steps, $\mathcal{E}'$ as the set of edges at time t , and $\mathbf{X}'$ as the node features at time t .

Definition 2 (CPD in Dynamic Graphs): The task of CPD in dynamic graphs aims to find a set of change points $\mathcal{T}^* = \{t_k^*\}_{k=1}^K$ from a graph series $\{\mathcal{G}'\}_{t=1}^T$. The change points split $[1, T]$ into contiguous segments such that $[1, T] = \bigcup_{k=1}^{K-1} [t_k^*, t_{k+1}^*]$, with $t_1^* = 1$ and $t_K^* = T$.

The node features $\mathbf{x}^t \in \mathbb{R}^4$ is an agent's 2-D position and velocity. The topology of graphs can change over time, including the addition and removal of nodes and edges. For convenience, this article groups all appearing nodes and assumes the node set is time-invariant, i.e., $\mathcal{V} = \bigcup_{t=1}^T \mathcal{V}'$ [19]. The edge set $\mathcal{E}'$ is essentially dynamic because the edge is defined by thresholding the distance between two agents, as will be described in Definition 3. Specifically, the curve of the number of edges over time is reported in Appendix D (supplementary material).

Under the distributed setting, the global monitor does not have direct access to agents' states. Each agent as a local detector has limited vision and shares limited messages.

Definition 3 (Distributed Setting for Emergence Detection): A qualified emergence detection method under the distributed setting should satisfy the following three conditions.

- 1) *Condition 1:* An agent only senses the states of other agents within a certain radius. Formally, the neighborhood of agent j at time t is defined as $\mathcal{N}_j^t = \{i : d_{ij}^t \leq \delta\}$, where d_{ij}^t is the Euclidean distance.
- 2) *Condition 2:* An agent j only communicates with its neighbors in $\mathcal{N}_j^t$.
- 3) *Condition 3:* The only message that an agent j shares with its neighbors or uploads to some monitor is its detecting score for emergence, i.e., a scalar $s_j^t \in [0, 1]$.

Inspired by Ranshous et al. [47], this article uses a dissimilarity function to calculate the detecting scores, and defines the criterion for CPD as follows.

Definition 4 (Criterion for CPD): Given a graph series and a dissimilarity function $d(\mathcal{G}, \mathcal{G}') \in \mathbb{R}_{\geq 0}$, a change point t is detected when $d(\mathcal{G}^t, \mathcal{G}^{t-1}) > c$ and $d(\mathcal{G}^t, \mathcal{G}^{t+1}) \leq c$ for some threshold c .

B. Motivation and Overview of HSTCL

The distributed setting stated in Definition 3 poses severe challenges to existing spatio-temporal modeling techniques and emergence detection methods:

- 1) *Conditions 1 and 3* state that the hidden vectors of agents are not shared. Thus, the common practice of stacking multiple GNN layers [48] to capture long-range dependence over multihop neighbors is inapplicable.
- 2) *Conditions 2 and 3* state that it is hard to reach consensus among agents via local communication within a limited time, because the communication graph changes constantly and it is not necessarily connected [49]. See Appendix A (supplementary material) for further demonstration.

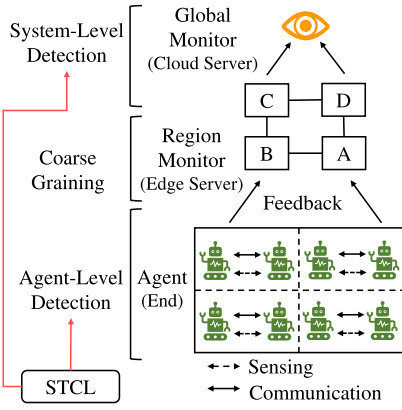


Fig. 2. Overview of HSTCL. HSTCL contains three hierarchies, agents, region monitors, and a global monitor, which can be conceptually implemented by the end-edge-cloud collaborative framework. Agents sense the states of neighbors, measure the change in relationships, and communicate with them to make agent-level detection. The detecting results are coarse-grained to region states, whose spatial patterns are used for system-level detection. Representations of agents and the system are learned via the STCL technique to support agent-level and system-level detection, respectively.

- 3) *Condition 3* states that the global monitor is unable to make global detection by utilizing agents' states in an end-to-end manner.

These challenges motivate the key design choice of HSTCL, that is, modeling the spatio-temporal dependency at different levels and aggregating the information hierarchically. An overview of HSTCL is shown in Fig. 2. It contains three hierarchies from bottom-up, *agents*, *region monitors*, and *a global monitor*. It can be conceptually implemented by the end-edge-cloud collaborative framework [50]. The area where all agents move is split into several connected regions. In each region, every agent senses the states of its neighbors and detects if its relationship with neighbors changes significantly. Each agent communicates with its neighbors to enhance the agent-level detecting results. The detecting results of agents within the same region are aggregated by the corresponding region monitor. The regional results are analyzed by the global monitor to make a system-level detection that is aware of emergence-related patterns.

Agent-level detection compresses an agent's local observations into a single detection score, while system-level detection unifies these scores to gain a global view. They are supported by agent-level and system-level representation learning, respectively. STEs are designed to capture nonlinear agent-to-agent and region-to-region relationships. Spatio-temporal consistency learning (STCL) strategy guides STEs to learn agents' and the system's representations that preserve both spatial and temporal consistency. Both agent-level and system-level STCL preserve the temporal consistency of representations within a time window. The former preserves the spatial consistency between each agent's and its neighbors' representations, and the latter preserves the spatial consistency between the system's and the regions' representations. The inconsistency in system representation serves as a detection signal for emergence. Formally, HSTCL can be described as

a three-step process, corresponding to its three-level structure

$$\begin{aligned} \mathbf{s}^{1:T} &= \text{AgentDetect}(\mathbf{X}^{1:T}) \in \mathbb{R}^{T \times |\mathcal{V}|} \\ \mathbf{y}^{1:T} &= \text{CoarseGrain}(\mathbf{s}^{1:T}) \in \mathbb{R}^{T \times M} \\ s_G^{1:T} &= \text{SystemDetect}(\mathbf{y}^{1:T}) \in \mathbb{R}^T. \end{aligned} \quad (1)$$

$\mathbf{s}^{1:T}$ is agent-level detecting scores, which are coarse-grained to M regions' states $\mathbf{y}^{1:T}$. $s_G^{1:T}$ is system-level detecting scores based on region states. Sections III-C and III-D will describe the process of HSTCL in detail.

C. Agent-Level Detection

1) *Spatio-Temporal Encoder*: To make online detection at time τ , each agent records its state and its neighbors' states in the last w time steps. Denoting $\tau(w) = \tau - w + 1$ as the initial step of the time window, these states are transformed to agent representations by the agent-level STE

$$\mathbf{h}_j^{\tau(w):\tau} = \text{STE}_A(\mathbf{x}_j^{\tau(w):\tau}, \{\mathbf{x}_i^t : i \in \mathcal{N}_j^t\}_{t=\tau(w)}^\tau) \in \mathbb{R}^{w \times D}. \quad (2)$$

Due to *Conditions 1 and 3* of the distributed setting, each agent cannot acquire their neighbors' latent representations. Thus, the popular design choice of integrated dynamic GNNs that model spatio-temporal entangled relations [51], [52] is inapplicable. Instead, this article adopts a stacked architecture composed of a spatial transformer and a temporal transformer [53], disentangling spatial and temporal dependency.

The spatial transformer models the relationship between an agent and its neighbors at each time step. The state of an agent $\mathbf{x}_j^t$ is first embedded as a hidden vector through a single-layer perceptron, i.e., $\mathbf{e}_j^t = \text{Emb}(\mathbf{x}_j^t)$. Then, a scaled dot-product attention mechanism [53] with a skip connection is applied to calculate the spatial representation

$$\mathbf{z}_j^t = \mathbf{e}_j^t + \sum_{i \in \mathcal{N}_j^t} \alpha_{ij}^t f_V(\mathbf{e}_i^t - \mathbf{e}_j^t) \quad (3)$$

where $\mathbf{e}_i^t - \mathbf{e}_j^t$ measures the spatial difference between agent j and its neighbor i in the latent space. It may capture nonlinear relations that are not fully reflected in quantities of the raw space, e.g., relative positions and velocities. f_V is a value function implemented by a linear mapping. $\alpha_{ij}^t = \text{softmax}(\{a_{ij}^t : i \in \mathcal{N}_j^t\})$ is the normalized attention score, with a_{ij}^t defined as

$$a_{ij}^t = \frac{1}{\sqrt{D}} f_Q(\mathbf{e}_j^t)^\top f_K(\mathbf{e}_i^t) \quad (4)$$

where f_Q and f_V are linear layers accounting for the query function and the key function, respectively. The spatial transformer does not require temporal embeddings of neighbors within a time window, which is desirable because the neighbors frequently change.

The temporal transformer is instantiated as a standard transformer [53], because it is powerful for sequential modeling, and it allows parallel execution, which is favorable for online detection. At its core is a temporal attention mechanism that maps spatial representations to temporal representations

$$\mathbf{h}_j^{\tau(w):\tau} = \text{softmax}\left(\frac{1}{\sqrt{D}} \mathbf{q}_j^{\tau(w):\tau} (\mathbf{k}_j^{\tau(w):\tau})^\top\right) \mathbf{v}_j^{\tau(w):\tau} \quad (5)$$

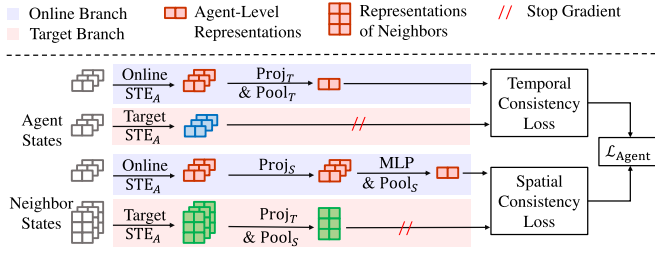


Fig. 3. Procedure of agent-level STCL.

where $\mathbf{q}_j^{\tau(w):\tau}$, $\mathbf{k}_j^{\tau(w):\tau}$, and $\mathbf{v}_j^{\tau(w):\tau}$ are query, key, and value vectors transformed from $\mathbf{z}^{\tau(w):\tau}$, respectively.

Disentangling the spatial and temporal information also makes the spatio-temporal encoder friendly to streaming data because the agent representations can be updated incrementally. As the time step τ increases by 1, only the current spatial representation $\mathbf{z}_j^{\tau+1}$ needs to be computed, while $\mathbf{z}_j^{\tau(w)+1:\tau}$ can be reused. For the temporal transformer, intermediate results like the unnormalized attention scores and the query vectors that only involve $\mathbf{z}_j^{\tau(w)+1:\tau}$ can be stored. The normalized attention scores and the temporal representations can be computed by further incorporating $\mathbf{z}_j^{\tau+1}$. Details can be found in Appendix C (supplementary material).

2) *Spatio-Temporal Consistency Learning*: Since the labels of emergence are rarely known a priori, this article proposes to train STE in a self-supervised manner by preserving the spatio-temporal consistency of the agent representations. STCL is inspired by an influential noncontrastive method called bootstrapping your own latent (BYOL) [54], which avoids explicit negative samples by aligning different views of the same sample encoded by asymmetric neural networks. BYOL is briefly introduced in Appendix A (supplementary material).

Unlike BYOL that uses a single objective, STCL disentangles the learning objectives of temporal consistency and spatial consistency since they characterize different aspects of the dynamic system. For each aspect, an online network and a target network with asymmetric network structures are designed to process different views of the same agent. These views are constructed by leveraging the intrinsic spatial and temporal relations within the data other than data augmentation that may damage the spatio-temporal semantics [45]. Given a view of some agent, the online network is trained to align the output of the target network for another view. The procedure of agent-level STCL is depicted in Fig. 3.

In Sections III-C2a–III-C2c, a symbol with a tilde stands for an element from the target branch, e.g., a vector $\tilde{\mathbf{h}}$ and a function $\tilde{f}$. Symbols without a tilde come from the online branch. A vector with a superscript t is called a transient representation at time t , e.g., $\mathbf{h}^t$. A vector with a superscript (τ) stands for a short-term representation within a time window $[\tau(w), \tau]$, e.g., $\mathbf{h}^{(\tau)}$.

a) *Temporal consistency loss*: When emphasizing the temporal consistency, the agent representation $\mathbf{h}_j^t$ is mapped to a temporal space via the temporal projection Proj_T . The resulting vectors $\mathbf{v}_j^t$ are reduced to a short-term representation

via a temporal pooling function Pool_T , mean pooling here

$$\mathbf{v}_j^t = \text{Proj}_T(\mathbf{h}_j^t), \quad \mathbf{v}_j^{(\tau)} = \text{Pool}_T(\mathbf{v}_j^{\tau(w):\tau}). \quad (6)$$

To ensure that the short-term representation is consistent with the transient representations, and thus capturing the tendency within the time window, this article minimizes the dissimilarity between them. The dissimilarity is defined as the complement of the cosine similarity

$$d(\mathbf{v}_j^{(\tau)}, \tilde{\mathbf{h}}_j^{(\tau)}) = \frac{1}{2} (1 - \cos(\mathbf{v}_j^{(\tau)}, \tilde{\mathbf{h}}_j^{(\tau)})). \quad (7)$$

Then, the temporal consistency loss is defined as the average temporal dissimilarity of all agents within the time window

$$\mathcal{L}_T = \frac{1}{w|\mathcal{V}|} \sum_{j \in \mathcal{V}} \sum_{t=\tau(w)}^{\tau} d(\mathbf{v}_j^{(\tau)}, \tilde{\mathbf{h}}_j^{(\tau)}). \quad (8)$$

b) *Spatial consistency loss*: When emphasizing the spatial consistency, $\mathbf{h}_j^t$ is mapped to a spatial space via the spatial projection Proj_S . To avoid disturbing the optimization of the temporal counterpart, this article further transforms the resulting vectors with a multilayer perceptron (MLP) to construct an asymmetric branch, i.e.,

$$\mathbf{n}_j^t = \text{Proj}_S(\mathbf{h}_j^t), \quad \mathbf{n}_j^{(\tau)} = \text{Pool}_T(\text{MLP}(\mathbf{n}_j^{\tau(w):\tau})). \quad (9)$$

By minimizing the dissimilarity between the short-term representation of each agent and its neighbors, the model learns to preserve spatial consistency, i.e.,

$$\mathcal{L}_S = \frac{1}{\kappa|\mathcal{V}|} \sum_{j \in \mathcal{V}} \sum_{i \in \mathcal{N}_j} d(\mathbf{n}_j^{(\tau)}, \tilde{\mathbf{n}}_i^{(\tau)}) \quad (10)$$

where $\mathcal{N}_j$ contains κ random neighbors from the temporal neighborhood $\cup_{t=\tau(w)}^{\tau} \mathcal{N}_j^t$. The sampling probability of a neighbor is proportional to its frequency.

As STE_A is responsible for the representation, while Proj_T and Proj_S are responsible for projections, they are simply implemented as MLPs. Mean pooling is adopted for Pool_T and Pool_S for simplicity, and more advanced spatial pooling [48], [55] and temporal pooling [56] methods are left for future work.

c) *Optimization*: Combining the temporal consistency loss and the spatial consistency loss, the overall loss for agent-level learning is

$$\mathcal{L}_{\text{Agent}} = \mathcal{L}_T + \mathcal{L}_S. \quad (11)$$

Directly minimizing the above loss will lead to collapsed representations [54]. To avoid this, the parameters Θ_A of the online branch are optimized by a gradient-based algorithm, e.g., Adam [57], while parameters $\tilde{\Theta}_A$ of the target branch are updated by exponential moving average [54]

$$\Theta_A \leftarrow \text{Opt}(\mathcal{L}_{\text{Agent}}, \Theta_A), \quad \tilde{\Theta}_A \leftarrow \eta \tilde{\Theta}_A + (1 - \eta) \Theta_A \quad (12)$$

where $\eta \in [0, 1]$ is a decay rate. The final Θ_A for emergence detection is obtained when the iterative process converges.

framework and in a single machine, respectively, corresponding to the potential real-world deployment and the actual implementation in our experiments for proof of concept. Their complexities mainly differ in agent-level detection.

In the end-edge-cloud collaborative implementation, agents accomplish the computation in parallel [60], [61]. Recall that D is the dimension of the hidden vector. For agent j , the time complexity of spatial encoding at time t is $O(|\mathcal{N}_j^t|D^2)$, and the time complexity of temporal encoding within a time window is $O(wD^2 + w^2D)$. Thus, the total complexity of spatio-temporal encoding is $O(wD^2 + w^2D + \sum_{t=\tau(w)}^{\tau} |\mathcal{N}_j^t|D^2)$. The time complexities for evaluating the temporal consistency loss and the spatial consistency loss are $O(wD^2)$ and $O(\kappa D^2)$, respectively. The time complexity of communication is $O(|\mathcal{N}_j^t|)$ at each time step. Therefore, at both the training and inference stages, the time complexity is linear with respect to the number of neighbors, which can be controlled by setting a budget. Hence, the distributed implementation scales well to large-scale systems.

In the single-machine implementation, the complexity of agent-level detection is relevant to the number of agents. The time complexity of spatial encoding at time t is $O(|\mathcal{E}^t|D^2)$, and the time complexity of temporal encoding within a time window is $O(|\mathcal{V}|(wD^2 + w^2D))$. Thus, the total complexity of spatio-temporal encoding is $O(|\mathcal{V}|(wD^2 + w^2D) + \sum_{t=\tau(w)}^{\tau} |\mathcal{E}^t|D^2)$. The time complexities for evaluating the temporal consistency loss and the spatial consistency loss are $O(w|\mathcal{V}|D^2)$ and $O(\kappa|\mathcal{V}|D^2)$, respectively. The time complexity of communication is $O(|\mathcal{E}^t|)$ at each time step. Therefore, at both the training and inference stages, the time complexity is linear with respect to the number of agents and the number of edges.

In both implementations, the system-level detection is conducted by the global monitor. The time complexity of system-level spatio-temporal encoding is $O(|\mathcal{RV}|(wD^2 + w^2D) + |\mathcal{RE}|wD^2)$. The complexities of evaluating system-level temporal consistency loss and spatial consistency loss are $O(wD^2)$ and $O(\kappa D^2)$, respectively. Thus, the complexity of system-level detection is linear with respect to the number of regions and the number of edges, which are generally irrelevant to the number of agents.

In Section IV-G, this article provides a running time analysis that matches the time complexity analysis to verify the scalability of HSTCL.

F. Characteristics of HSTCL

HSTCL is characterized by the following features.

- 1) By hierarchically aggregating agent-level detecting results, HSTCL can capture emergence-related spatial patterns ignored by DETect, where agents' feedback is summed up indiscriminately.
- 2) Thanks to the spatio-temporal disentangled architecture, STE can capture agents' nonlinear relationships under the distributed setting, where popular designs of spatio-temporal integrated GNNs are infeasible.
- 3) STCL preserves the spatio-temporal consistency within both agent-level and system-level representations. Com-

TABLE I
STATISTICS OF DATASETS

Datasets	Flock	Pedestrian	Traffic
# Agents	150	382	2,522
Shape of grid	51×51	40×40	841×841
# Simulation steps	50,000	50,000	60,000
# Evaluation steps	1,000	1,000	1,200
# Change points	10	10	≈ 10

pared with BYOL, it avoids potentially harmful data augmentations, and can handle multiple objectives in a disentangled way. It is free of negative samples, significantly reducing the computational cost for spatio-temporal data.

- 4) HSTCL is flexible in integrating other deep learning methods as long as they satisfy the distributed setting described in Definition 3. First, existing nondistributed AD and CPD methods can be transformed into distributed detectors by adapting them to agent-level and system-level detection separately. Second, STEs can be implemented by other spatio-temporally disentangled GNNs. Third, STCL can be replaced by other self-supervised training schemes like contrastive learning. Finally, other dissimilarity functions and detection criteria can be adopted for emergence detection.

IV. EXPERIMENTS

A. Datasets

For a fair comparison, this article follows DETect [7] and adopts three simulation environments implemented by NetLogo [62] to generate data. These simulators are equipped with well-known yet hard-to-detect emergent phenomena. In all simulators, agents move in a 2-D-bounded world composed of patches. The resulting datasets are briefly described as follows.

- 1) *Flock* [63]: Each agent is a bird. The emergence is the flocking behavior. The objective measure of emergence is the number of patches that contain no birds.
- 2) *Pedestrian* [11]: Each agent is a pedestrian walking either from left to right or in the opposite position. The emergence is the counter-flow. The objective measure of emergence is the number of lanes formed by pedestrians.
- 3) *Traffic* [7]: Each agent is a car running on the road net of Manhattan, New York City. The road net contains 6657 junctions and 77 569 road segments. The cars are routed by a routing engine GraphHopper [64] based on real-world car records. The emergence happens when a significant number of streets get congested. Thus, the objective measure of emergence is the number of congested road segments.

On the Flock and Pedestrian datasets, real-world data is unavailable. Thus, reasonable behavioral rules are designed for agents. On the Traffic dataset, the real-world data is combined with simulation rules to mimic agents' behaviors. Visualizations of emergent behaviors on all datasets and more details of the Traffic dataset are shown in Appendix D

(supplementary material). A summary of important statistics of the datasets is shown in Table I. Each dataset contains 20 times of simulations, with five times as the training set, five times as the validation set, and the rest as the testing set. Following O’toole et al. [7], the objective measure is evaluated every 50 steps. The ground truth change points are labeled by running offline CPD algorithms provided by ruptures [65]. Offline CPD algorithms have access to the whole series, and thus the labeled change points are more reliable and accurate. The results are checked manually. It turns out that change points make up no more than 1% of all evaluation steps. The severe imbalance between change points and normal points further increases the challenge of emergence detection.

A natural question arises: Why not examine the performance of detectors on real-world datasets? To the best of our knowledge, there is no benchmark based on purely real-world data. Currently, collecting such data can be difficult in three aspects: 1) the emergent behavior should be properly understood because labeling emergence formation and evaporation requires prior knowledge; 2) contiguously recording all agents’ states for a long period can be challenging to the sensing devices; and 3) due to some unavailability issues of real data, it is hard to ensure the diversity. The simulators can overcome these limitations, and they may generate potentially diverse and challenging data that are uneasy to collect in practice, helping to evaluate the detector’s performance more comprehensively. We will try to construct qualified real-world datasets in the future.

B. Evaluation Metrics

Due to the unpredictability of emergence, it can be difficult to detect the exact change points. The formation or evaporation of emergence can happen in a short period rather than a specific time step. Therefore, it is reasonable to accept more than one detection around a true change point in practice. In this article, the detections within a given tolerance θ are regarded as one true positive (TP), while the rest detections are regarded as false positive (FP), i.e.,

$$\begin{aligned} \text{TP} &= |\{t^* \in \mathcal{T}^* : \exists t \in \widehat{\mathcal{T}}, \text{ s.t. } |t - t^*| \leq \theta\}| \\ \text{FP} &= |\{t \in \widehat{\mathcal{T}} : \forall t^* \in \mathcal{T}^*, |t - t^*| > \theta\}| \end{aligned}$$

where $\mathcal{T}^*$ and $\widehat{\mathcal{T}}$ are the set of true change points and the set of detected ones, respectively. This article sets $\theta = 20$ for all datasets. Defining the precision $\text{Prec} = (\text{TP}/(\text{TP} + \text{FP}))$ and the recall rate $\text{Rec} = (\text{TP}/|\mathcal{T}^*|)$, the $F1$ score can be computed as

$$F1 = \frac{2 \times \text{Prec} \times \text{Rec}}{\text{Prec} + \text{Rec}}.$$

The $F1$ score measures the overall accuracy of CPD. This article further uses the covering metric [66], [67] to measure the overlapping degree between the ground truth segments and the detected segments. Let $\mathcal{A}^*$ be the set of ground truth segments $\mathcal{I}^*$, with a similar definition for $\widehat{\mathcal{A}}$ and $\widehat{\mathcal{I}}$. The covering metric is defined as

$$\text{Cover}(\mathcal{A}^*, \widehat{\mathcal{A}}) = \frac{1}{T} \sum_{\mathcal{I}^* \in \mathcal{A}^*} |\mathcal{I}^*| \cdot \max_{\widehat{\mathcal{I}} \in \widehat{\mathcal{A}}} J(\mathcal{I}^*, \widehat{\mathcal{I}})$$

where $J(\mathcal{I}^*, \widehat{\mathcal{I}}) = |\mathcal{I}^* \cap \widehat{\mathcal{I}}|/|\mathcal{I}^* \cup \widehat{\mathcal{I}}|$ is the Jaccard index [68] measuring the overlapping degree between two segments.

In the original paper of DETect [7], the detecting performance is quantitatively evaluated by checking if the number of detected events is significantly larger during the emergent periods than the nonemergent periods. Nonetheless, the deviation between detected change points and the ground truth is not assessed. Therefore, this article hopes to fill the gap by introducing the two metrics, and push the current research toward more timely emergence detection.

C. Baselines

To demonstrate the effectiveness of our method, this article compares it with DETect and some state-of-the-art deep learning methods in closely related fields, including dynamic network CPD method sGNN [20], time series CPD method TS-CPP [69], time series AD method GDN [23], and graph-level AD method OCGTL [36]. Advanced techniques like GNNs and contrastive learning are used in these methods. They are adapted to our framework at both agent-level and system-level detection. They are renamed with the suffix “+H,” short for hierarchical framework.

- 1) *DETect*: A decentralized method for online emergence detection. Each agent detects the change in relationships between its neighbors and itself via a linear model. The detecting results are aggregated to make a global decision.
- 2) *sGNN+H*: A dynamic network CPD method that uses siamese GNNs to learn the graph similarity between two graph snapshots. A top- k pooling module is applied to summarize the node-wise distances in the latent space into a graph similarity.
- 3) *TS-CPP+H*: A time series CPD method based on contrastive learning. Temporal convolutional networks [70] are used for time series encoding, and the similarity between two contiguous time segments is used as the indicator for CPD.
- 4) *GDN+H*: A GNN-based method for multivariate time series AD. It uses graph attention to capture the relationships between sensors and defines the maximum deviation score for AD.
- 5) *OCGTL+H*: A graph-level AD method that combines one-class classification and neural transformation learning [71], an advanced self-supervised learning technique.

The codes of all baselines are publicly available. The code of DETect¹ is directly applied to our experiments. The code of sGNN,² TS-CPP,³ GDN,⁴ and OCGTL⁵ are adapted to our framework. Implementation details of HSTCL are described in Appendix D (supplementary material). The threshold c in Definition 4 is decided by maximizing the $F1$ score on the validation set.

¹<https://github.com/viveknallur/DETectEmergence/>

²<https://github.com/dsulem/DynNet>

³<https://github.com/cruiseresearchgroup/TSCPP2>

⁴<https://github.com/d-ailin/GDN>

⁵<https://github.com/boschresearch/GraphLevel-AnomalyDetection>

TABLE II

DETECTING PERFORMANCE OF DIFFERENT METHODS. BOTH THE MEAN VALUE AND THE STANDARD DEVIATION ARE REPORTED. THE BEST RESULTS ARE IN **BOLD**. THE IMPROVEMENTS ARE SIGNIFICANT (p -VALUE < 0.05). THE RELATIVE IMPROVEMENTS ARE COMPUTED WITH RESPECT TO DETECT

Datasets	Flock		Pedestrian		Traffic	
Metrics	F1↑	Cover↑	F1↑	Cover↑	F1↑	Cover↑
TS-CPP+H	0.7003± 0.0029	0.6444± 0.0148	0.7105± 0.0260	0.6720± 0.0320	0.3673± 0.0370	0.5315± 0.0313
GDN+H	0.6755± 0.0441	0.6902± 0.0254	0.7181± 0.0319	0.7123± 0.0132	0.3473± 0.0067	0.5276± 0.0040
OCGTL+H	0.7092± 0.0301	0.7299± 0.0437	0.9248± 0.0207	0.8854± 0.0134	0.3674± 0.0379	0.5713± 0.0194
sGNN+H	0.7109± 0.0082	0.7372± 0.0050	0.7061± 0.0019	0.6458± 0.0031	0.3611± 0.0257	0.5329± 0.0105
DETECT	0.4862± 0.0507	0.6559± 0.0284	0.2064± 0.0807	0.4408± 0.0416	0.3479± 0.0875	0.5479± 0.0558
HSTCL	0.7757± 0.0026	0.7810± 0.0116	0.9352± 0.0096	0.9235± 0.0107	0.3928± 0.0235	0.5872± 0.0093
Improvement	+59.54%	+19.07%	+353.10%	+109.51%	+12.91%	+7.17%

TABLE III

ABLATION STUDY. BOTH THE MEAN VALUE AND THE STANDARD DEVIATION ARE REPORTED. THE BEST RESULTS OF AGENT-LEVEL AND SYSTEM-LEVEL DETECTION ARE IN **BOLD**. THE IMPROVEMENTS ARE SIGNIFICANT (p -VALUE < 0.05)

Datasets	Flock		Pedestrian		Traffic	
Metrics	F1↑	Cover↑	F1↑	Cover↑	F1↑	Cover↑
HSTCL _{Agent-S}	0.6351± 0.0451	0.6962± 0.0438	0.7627± 0.0556	0.7430± 0.0361	0.3189± 0.0265	0.5319± 0.0106
HSTCL _{Agent-T}	0.6264± 0.0271	0.6847± 0.0275	0.7773± 0.0119	0.7490± 0.0250	0.3197± 0.0361	0.5227± 0.0164
HSTCL _{Agent}	0.6705± 0.0114	0.6960± 0.0168	0.7965± 0.0163	0.7601± 0.0108	0.3538± 0.0027	0.5445± 0.0090
HSTCL _S	0.7155± 0.0317	0.7516± 0.0169	0.9220± 0.0142	0.9089± 0.0199	0.3690± 0.0266	0.5755± 0.0361
HSTCL _T	0.7564± 0.0620	0.7680± 0.0310	0.9278± 0.0122	0.9082± 0.0187	0.3735± 0.0233	0.5880± 0.0219
HSTCL _{Self}	0.6413± 0.0303	0.6899± 0.0319	0.7732± 0.0128	0.7211± 0.0325	0.3271± 0.0227	0.5382± 0.0088
HSTCL	0.7757± 0.0026	0.7810± 0.0116	0.9352± 0.0096	0.9235± 0.0107	0.3928± 0.0235	0.5872± 0.0093

D. Comparison With Baselines

The detecting performance of different methods is shown in Table II. The metrics of DETECT are relatively low on all datasets, showing that emergence detection can be difficult for traditional methods even on the simulation datasets. Deep learning methods generally outperform DETECT in both metrics, and HSTCL achieves the highest performance. The results verify the superiority of our framework, which can capture emergence-related spatial patterns and model nonlinear spatio-temporal dynamics. sGNN+H is relatively poor on the Pedestrian dataset. It ignores the temporal dependence between adjacent graph snapshots and simply averages the similarities within the time window. HSTCL captures the temporal dependence via the STEs and learns a short-term representation of the system within the time window, which can better reflect the system-level variation in the latent space. GDN+H calculates the detecting score based on next-step prediction error, which can be sensitive to the noise in the states. HSTCL calculates the score based on the consistency of representations, which is resistant to potential noise. HSTCL jointly preserves spatial and temporal consistency, and thus outperforms TS-CPP+H which ignores spatial consistency and OCGTL+H which ignores temporal consistency.

E. Ablation Study

Some variants of HSTCL are introduced to verify the necessity of capturing emergence-related patterns and modeling agents' relationships with neighbors. HSTCL_{Agent} removes system-level detection. HSTCL_{Self} makes agent-level detection

without modeling the spatial relationships, i.e., removing the spatial encoder and training without the spatial consistency loss. The results are shown in Table III.

1) *Effect of System-Level Detection*: Without system-level detection, the average $F1$ score and covering metric of HSTCL_{Agent} decrease by 0.0943 and 0.0970, respectively. The results verify that the spatial patterns of agent-level detecting results help with emergence detection.

To see what patterns are captured by HSTCL, a case study is conducted on the Traffic dataset. Fig. 5 visualizes the congesting states of the road net and the region states when the network-level congestion forms. Fig. 5(a) shows that congested road segments constitute a connected subnetwork with a diameter of 80, accounting for more than (1/3) of the diameter of the road net. The phenomenon confirms the emergence of widespread congestion. Such emergence-related pattern is almost faithfully reflected in region states. The results also show that HSTCL can detect the emergence of widespread congestion even when the traffic flow is not provided.

2) *Effect Of Agent-Level Detection*: Compared with HSTCL_{Agent}, the average $F1$ score and the covering metric of HSTCL_{Self} decrease by 0.0264 and 0.0171, respectively. The results show that modeling the nonlinear relationship between an agent and its neighbors is indispensable for agent-level detection. Note that HSTCL_{Agent} is better than DETECT on the Flock and Pedestrian datasets, and on par with it on the Traffic dataset, which again verifies the effectiveness of agent-level spatio-temporal modeling.

To see how HSTCL_{Agent} captures the relationships, this article conducts a case study on the Flock dataset. Inspired

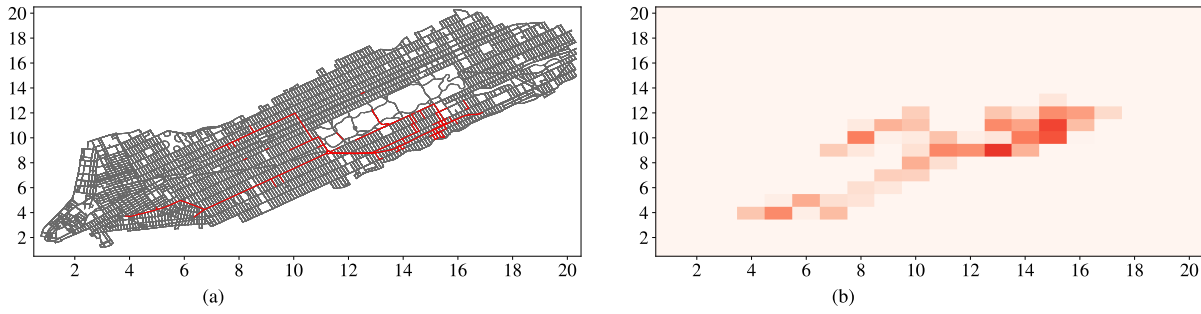


Fig. 5. Case study of spatial patterns on the Traffic dataset. In (a), each line segment represents a road segment. Segments in red are congested, while those in gray are normal. In (b), each grid represents a region state. The darker the color, the more agent-level detections are collected. Best viewed in color.

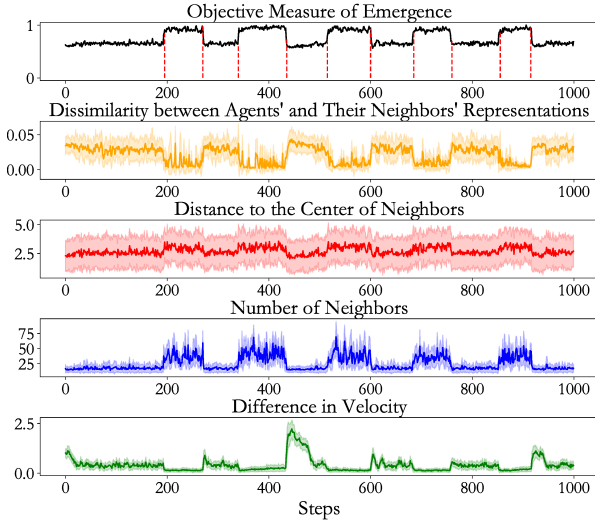


Fig. 6. Case study of agents' relationships on the Flock dataset. The variation curves of the objective measure, agents' dissimilarity in latent space, and agents' relationships with respect to three indicators are visualized. Best viewed in color.

by O'toole et al. [7], three indicators are used to measure the relationships with respect to agents' states, including the distance to the center of neighbors, the number of neighbors, and the difference between an agent's velocity and its neighbors' average velocity. The objective measure of emergence is set as a reference. Agents' and their neighbors' dissimilarity in representations stands for relationships in the latent space. The variation curves of the aforementioned metrics are shown in Fig. 6. As expected, during the emergent period of flocking, birds get crowded, and thus, the distances are smaller, the neighbor count increases, and the difference in velocity decreases. Unexpectedly, the difference in velocity between 400 and 600 steps is larger than those in other periods. This anomalous phenomenon may be attributed to the flocking simulation rules. Birds are set to align during the emergent period and move randomly during the nonemergent period. The second emergent period is relatively long and thus the subsequent nonemergent period witnesses a sharp increase in velocity difference. The unstable behavior of the last metric shows that a single metric may not always be reliable for indicating emergence.

Different metrics present different patterns, yet these patterns are approximately captured by the latent representations. The tendency of the dissimilarity curve also agrees with that of the objective measure. These results show that our method can

somehow comprehensively capture the relationships defined by some intuitive metrics with respect to agents' states.

3) *Effect of Spatial and Temporal Consistency Losses:* To validate the effectiveness of STCL for both agent-level and system-level detection, this article introduces variants of $HSTCL_{Agent}$ and $HSTCL$ trained with only the spatial or the temporal consistency loss. The resulting methods are denoted as $HSTCL_{Agent-S}$, $HSTCL_{Agent-T}$, $HSTCL_S$, and $HSTCL_T$, respectively. As shown in Table III, removing any term in the loss function will lead to degenerated performance in most cases. For example, $HSTCL_{Agent-T}$ removes the agent-level spatial consistency loss, and noticeable drops in both metrics can be observed on all datasets. The results verify that inconsistency in spatial relation is an accurate indicator of emergence. Similarly, $HSTCL_S$ removes the system-level temporal consistency loss and both metrics decrease. The results verify that temporal inconsistency of the whole system helps to detect the emergent behavior. Thus, the spatial consistency loss and temporal consistency loss are complementary to learning discriminative representations for emergence detection.

F. Hyperparameter Analysis

1) *Effect of Region Granularity:* The area where agents move is split into many regions for system-level detection. The granularity of regions decides how many details are preserved for global analysis. To study the effect of region granularity, this article trains the system-level detector on the Traffic dataset under several $N \times N$ grids, with $N \in \{5, 10, 15, 20, 25, 30\}$. The results are shown in Fig. 7. The $F1$ score is lowest when $N = 5$. Maybe coarsening too much will result in inadequate information that cannot support accurate detection. The $F1$ score and covering metric increase on the whole as N grows but start to decrease at $N = 20$. An exception is that the covering metric decreases at $N = 10$. The $F1$ score is the harmonic mean of the precision and the recall rate. Since the threshold c for determining a change point is searched by maximizing the $F1$ score on the validation set, it is possible that the precision or the recall rate increases while the other metric decreases, leading to the growth of the $F1$ score and the drop of the covering metric. Some examples are provided in Appendix D (supplementary material). When N is too large, the regions are too small to collect sufficient feedback from agents and present stable spatial patterns. Thus, our method achieves the highest performance for $N = 20$ with a moderate computational cost.

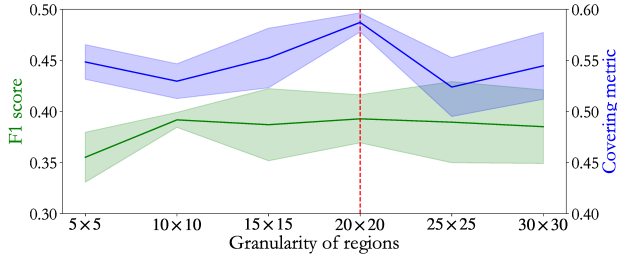


Fig. 7. Effect of region granularity on the Traffic dataset. The dashed vertical line indicates the best results. Best viewed in color.

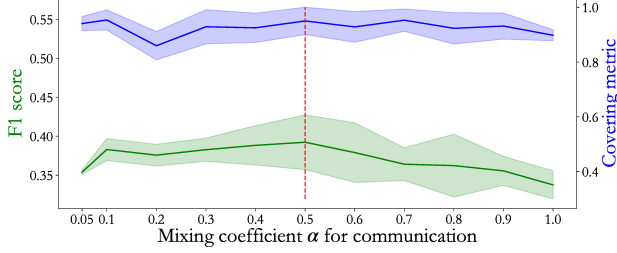


Fig. 8. Effect of agents' communication on the Traffic dataset. The dashed vertical line indicates the best results. Best viewed in color.

2) *Effect of Mixing Coefficient for Communication:* Agents communicate with neighbors to make agent-level detections. The mixing coefficient α in (13) balances the importance of an agent's current observation and its neighbors' detecting scores. α is set to 0.05 to keep consistent with DETect. To see how α affects the detecting accuracy, this article evaluates HSTCL_{Agent} on the Traffic dataset with α ranging from 0.1 to 1. As shown in Fig. 8, the $F1$ score and covering metric can be promoted by increasing α , i.e., assigning a larger weight to agents' current observation. However, when $\alpha = 1$, i.e., agents ignore the detecting scores of their neighbors, the $F1$ score drops significantly, verifying the necessity of communication. The results show the possibility of tuning α to improve the detecting accuracy. Furthermore, α can be personalized and adaptive over time. Optimizing the choices of α is left for future work.

3) *Effect of Window Size for Emergence Detection:* The window size w is the temporal scope of emergence detection. Like the granularity of regions, there is a tradeoff between precision and efficacy with respect to w . It is set to 10 for agent-level detection, since the objective measure of emergence is evaluated every 50 steps and the states of agents are downsampled every 5 steps. In this way, agents can make relatively accurate and timely detections. However, performance degeneration is observed for system-level detection with the same window size. It is conjectured that system-level detection needs a larger temporal scope with coarse-grained spatial information. The effect of w is studied on the Pedestrian dataset, with $w \in \{10, 20, \dots, 100\}$. As shown in Fig. 9, the $F1$ score generally increases as w grows, while the covering metric peaks at $w = 40$. Although the $F1$ score peaks at $w = 60$, both the running time and the memory consumption will increase, as depicted in Fig. 10. Since online detection is sensitive to the computational cost, w is set to 40 for system-level detection to achieve a good tradeoff between accuracy and efficiency. This choice is also practical since the global monitor generally has a larger capacity than a single agent.

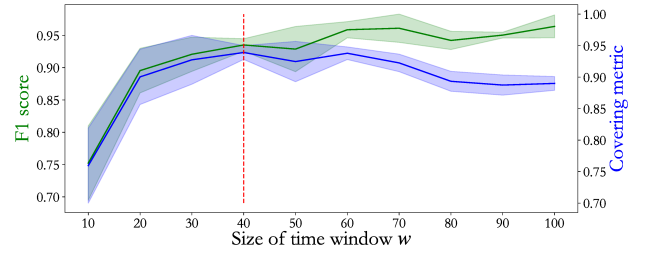


Fig. 9. Effect of window size in system-level detection on the Pedestrian dataset. Best viewed in color.

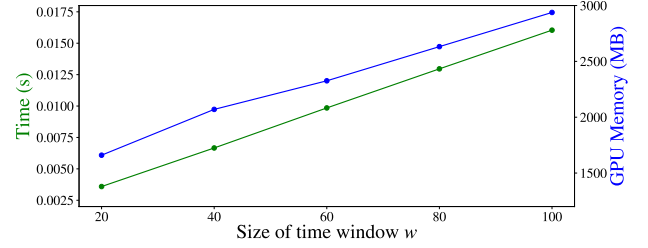


Fig. 10. Running time and GPU memory consumption of system-level detection with respect to w on the Pedestrian dataset.

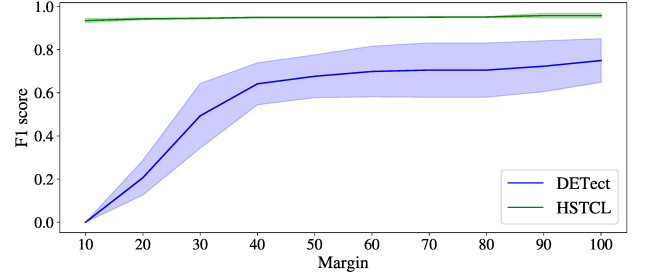


Fig. 11. Effect of threshold θ on the Pedestrian dataset. Best viewed in color.

4) *Effect of Threshold θ :* In the experiments, the threshold θ is set to 20 on all datasets for fairly comparing the detecting precision of different methods. Apparently, the $F1$ scores are affected by the choices of θ . This article evaluates the $F1$ scores of HSTCL and DETect on the Pedestrian dataset for $\theta \in \{10, 20, \dots, 100\}$, and the results are shown in Fig. 11. For both methods, the $F1$ score grows approximately as θ increases, because a larger tolerance allows to include more detected change points. On the whole, HSTCL consistently achieves higher detecting precision than DETect for all θ , yet the difference narrows down as θ increases. Besides, the variance of DETect's $F1$ scores tends to grow up for a larger θ , while HSTCL preserves a considerably smaller variance, showing that the performance of HSTCL is more stable. In practice, a relatively smaller threshold is more favorable, because detected change points with smaller displacement help to detect the emergence more timely.

To further differentiate the detecting quality of DETect and HSTCL, this article visualizes their detected change points on the Pedestrian dataset. As shown in Fig. 12, DETect fails to detect change points in several periods, and the detected points are relatively far from the nearest ground truth. By contrast, HSTCL successfully detects all change points with significantly smaller deviations.

5) *Sensitivity Analysis:* It is crucial to analyze the sensitivity of HSTCL with respect to the initial parameters

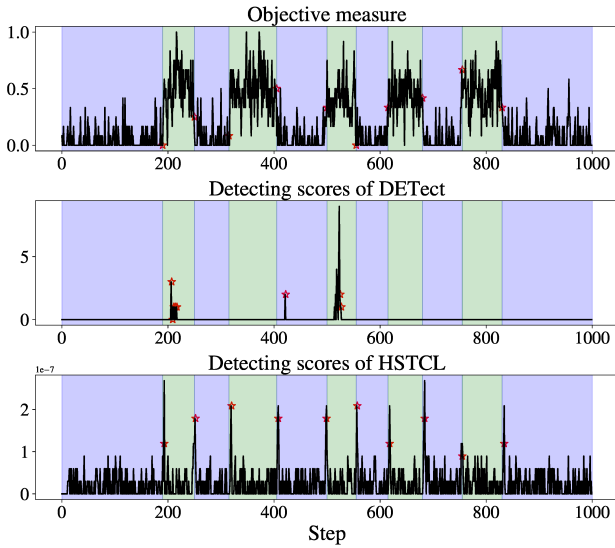


Fig. 12. (Top) Visualization of the ground truth change points, and change points detected by (middle) DETect and (bottom) HSTCL on the Pedestrian dataset. The variation curves are in black. The change points are marked as stars. The periods of emergence are in green. The normal periods are in purple. Best viewed in color.

for instructing its real-world applications. As reported in Tables II and III, the standard deviations for both agent-level and system-level detection of HSTCL are relatively small, which indicates that HSTCL is relatively robust to the stochastic nature of deep learning, including the weight initialization and the training process. In Section IV-Fa–d), this article analyzes the effect of region granularity $N \times N$, mixing coefficient α for communication, window size w , and the threshold θ for evaluating metrics. To summarize, HSTCL is sensitive to $N \times N$, α , and w , but is relatively stable with respect to θ . In practice, one can use the historical data as a validation set to determine essential parameters and adjust these parameters when there is a distribution shift of the online data. Specifically, α can be tuned without retraining the agent-level model. Adjusting N may not affect the deployment of edge monitors because one monitor can collect agents' feedback from several regions, but the system-level model should be updated. Adjusting w requires updating the agent-level model and the system-level model sequentially. When the system is nonstationary, it can be analyzed on multiple time scales simultaneously to achieve timely detection with relatively low cost [72].

G. Running Time Analysis

To study the efficiency of all methods for online emergence detection, this article reports their average running time per step for agent-level detection and system-level detection in Fig. 13. Although DETect is the most efficient running time on the Flock dataset which contains only 150 agents, its running time grows steeply with respect to the number of agents and peaks on the Traffic dataset which contains 2522 agents. Such inefficiency may be due to its concrete implementation in NetLogo. Among deep learning methods within our framework, GDN+H and sGNN+H are faster than others because of their simplicity in spatio-temporal modeling. The overall running time of

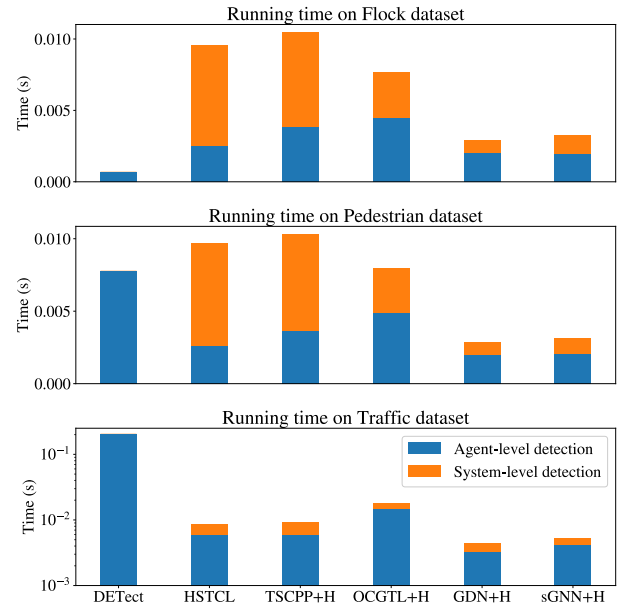


Fig. 13. Runtime of different methods on all datasets. Best viewed in color.

HSTCL, TSCPP+H, and OCGTL+H are comparable. However, OCGTL+H takes more time in agent-level detection than other methods, which may be due to its computationally costly design of transformation learning. Notably, on the large-scale Traffic dataset, the efficiency of HSTCL becomes closer to sGNN+H, which demonstrates the scalability of HSTCL.

On the Traffic dataset where agents are significantly more than regions, the overall running time is dominated by agent-level detection. On the Flock and Pedestrian datasets where regions are more than agents, system-level detection dominates the running time for HSTCL and TSCPP+H. For GDN+H, OCGTL+H, and sGNN+H, agent-level detection takes more time. This may be attributed to the simplicity or ignorance of temporal modeling, and that agent graphs are denser than region graphs. Note that the running time of all methods is evaluated on a single machine. In practice, agent-level detection can be accomplished by each agent in parallel. Thus, it is expected that all distributed detection methods scale well as the number of agents grows.

V. CONCLUSION

This article proposes a hierarchical framework named HSTCL for emergence detection in CAS under the distributed setting. By aggregating agent-level detecting results from the bottom-up, HSTCL learns a system representation that captures emergence-related patterns. Nonlinear relationships between agents and their neighbors are encoded in agent representations through STE. These representations are learned in a self-supervised manner by preserving the spatio-temporal consistency. HSTCL surpasses the traditional methods and deep learning methods on three datasets with well-known yet hard-to-detect emergent phenomena. HSTCL is flexible in incorporating deep learning methods from graph-level CPD and AD for effective emergence detection.

In the future, HSTCL can be extended from three dimensions: data, problems, and methods. On the data dimension, we plan to acquire real-time data streams from devices in

the Internet of Things to test HSTCL's effectiveness in live environments. Besides, allowing the addition and removal of agents will make the simulators more realistic and bring extra challenges to dynamic graph learning [51]. It is also promising to utilize simulators based on large language models [73] to promote the quality of simulation data and make the evaluation results more convincing. These simulators can better incorporate domain knowledge, and recover complex agent behaviors beyond predefined simulation rules. When real-world observations of agent states are sparse and incomplete, graph learning methods can be developed to complement missing information and possible auxiliary information can be leveraged to assist the detection [74]. When the systems contain heterogeneous agents, category-aware STEs and learning strategies can be designed to modeling heterogeneous dynamics and interacting patterns [75]. On the problem dimension, it is more realistic to consider partially missing messages and time delays in agents' communication, which will trigger the research of more robust detectors. On the method dimension, distributed GNNs and advanced CPD methods can further boost the performance of emergence detection. Besides, it is promising to develop graph learning methods that can capture emergence-related higher order structures of a system.

REFERENCES

- [1] O. Artime and M. De Domenico, "From the origin of life to pandemics: Emergent phenomena in complex systems," *Phil. Trans. Roy. Soc. A, Math., Phys. Eng. Sci.*, vol. 380, no. 2227, Jul. 2022, Art. no. 20200410.
- [2] S. Kalantari, E. Nazemi, and B. Masoumi, "Emergence phenomena in self-organizing systems: A systematic literature review of concepts, researches, and future prospects," *J. Organizational Comput. Electron. Commerce*, vol. 30, no. 3, pp. 224–265, Jul. 2020.
- [3] E. O'Toole, "Decentralised detection of emergence in complex adaptive systems," Ph.D. dissertation, School Comput. Sci. Statist., Trinity College, Dublin, Ireland, 2016.
- [4] J. Fromm, "Types and forms of emergence," 2005, *arXiv:nlin/0506028*.
- [5] M. A. Bedau, "Weak emergence," *Phil. Perspect.*, vol. 11, pp. 375–399, Jan. 1997.
- [6] G. Zeng et al., "Percolation-based health management of complex traffic systems," *Frontiers Eng. Manage.*, vol. 8, no. 4, pp. 557–571, Dec. 2021.
- [7] E. O'toole, V. Nallur, and S. Clarke, "Decentralised detection of emergence in complex adaptive systems," *ACM Trans. Auton. Adapt. Syst.*, vol. 12, no. 1, pp. 1–31, 2017.
- [8] S. Aminikhanghahi and D. J. Cook, "A survey of methods for time series change point detection," *Knowl. Inf. Syst.*, vol. 51, no. 2, pp. 339–367, Sep. 2016.
- [9] J. Gignoux, G. Chérel, I. D. Davies, S. R. Flint, and E. Lateltin, "Emergence and complex systems: The contribution of dynamic graph theory," *Ecol. Complex.*, vol. 31, pp. 34–49, Sep. 2017.
- [10] M. Mnif and C. Müller-Schloer, "Quantitative emergence," in *Organic Computing—A Paradigm Shift for Complex Systems*. Cham, Switzerland: Springer, 2011, pp. 39–52.
- [11] J. Procházka and K. Olševičová, "Monitoring lane formation of pedestrians: Emergence and entropy," in *Proc. Asian Conf. Intell. Inf. Database Syst.*, 2015, pp. 221–228.
- [12] Q. Liu, M. He, D. Xu, N. Ding, and Y. Wang, "A mechanism for recognizing and suppressing the emergent behavior of UAV swarm," *Math. Problems Eng.*, vol. 2018, pp. 1–14, Sep. 2018.
- [13] J. Luo, J. Xin, G. Binghui, H. Zheng, W. Wu, and W. Lv, "Dynamic model and crowd entropy measurement of crowd intelligence system," (in Chinese), *Scientia Sinica Informationis*, vol. 52, no. 1, pp. 99–110, 2022.
- [14] D. Fisch, M. Jänicke, B. Sick, and C. Müller-Schloer, "Quantitative emergence - a refined approach based on divergence measures," in *Proc. 4th IEEE Int. Conf. Self-Adaptive Self-Organizing Syst.*, Sep. 2010, pp. 94–103.
- [15] A. K. Seth, "Measuring emergence via nonlinear Granger causality," in *Proc. ALIFE*, vol. 2008, 2008, pp. 545–552.
- [16] R. L. Grossman et al., "Discovering emergent behavior from network packet data: Lessons from the angle project," in *Next Generation of Data Mining*. Boca Raton, FL, USA: CRC Press, 2008, pp. 267–284.
- [17] M. A. Niazi and A. Hussain, "Sensing emergence in complex systems," *IEEE Sensors J.*, vol. 11, no. 10, pp. 2479–2480, Oct. 2011.
- [18] E. Santos and Y. Zhao, "Automatic emergence detection in complex systems," *Complexity*, vol. 2017, pp. 1–24, Jan. 2017.
- [19] S. Huang, S. Coulombe, Y. Hitti, R. Rabbany, and G. Rabusseau, "Laplacian change point detection for single and multi-view dynamic graphs," *ACM Trans. Knowl. Discovery Data*, vol. 18, no. 3, pp. 1–32, Apr. 2024.
- [20] D. Sulem, H. Kenlay, M. Cucuringu, and X. Dong, "Graph similarity learning for change-point detection in dynamic networks," *Mach. Learn.*, vol. 113, no. 1, pp. 1–44, Jan. 2024.
- [21] X. Ma et al., "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012–12038, Dec. 2023.
- [22] A. D. Pazho, G. A. Noghre, A. A. Purkayastha, J. Vempati, O. Martin, and H. Tabkhi, "A survey of graph-based deep learning for anomaly detection in distributed systems," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 1, pp. 1–20, Jun. 2024.
- [23] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, May 2021, pp. 4027–4035.
- [24] Y. Zheng et al., "Correlation-aware spatial-temporal graph learning for multivariate time-series anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11802–11816, Sep. 2024, doi: 10.1109/TNNLS.2023.3325667.
- [25] Y. Wang, A. Chakrabarti, D. Sivakoff, and S. Parthasarathy, "Fast change point detection on dynamic social networks," in *Proc. 26th Int. Joint Conf. Artif. Intell.*, Aug. 2017, pp. 2992–2998.
- [26] S. Huang, Y. Hitti, G. Rabusseau, and R. Rabbany, "Laplacian change point detection for dynamic graphs," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 349–358.
- [27] J. Cheng, M. Chen, M. Zhou, S. Gao, C. Liu, and C. Liu, "Overlapping community change-point detection in an evolving network," *IEEE Trans. Big Data*, vol. 6, no. 1, pp. 189–200, Mar. 2020.
- [28] M. Li et al., "Guest editorial: Deep neural networks for graphs: Theory, models, algorithms, and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 4367–4372, Apr. 2024.
- [29] X. Zhang et al., "VGGM: Variational graph Gaussian mixture model for unsupervised change point detection in dynamic networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4272–4284, 2024.
- [30] P. Jiao et al., "Generative evolutionary anomaly detection in dynamic networks," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12234–12248, May 2023.
- [31] P. Jiao, T. Li, H. Wu, C.-D. Wang, D. He, and W. Wang, "HB-DSBM: Modeling the dynamic complex networks from community level to node level," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 8310–8323, Jun. 2023.
- [32] J. Li, R. Zheng, H. Feng, M. Li, and X. Zhuang, "Permutation equivariant graph framelets for heterophilous graph learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11634–11648, Sep. 2024.
- [33] Y. Zheng et al., "Toward graph self-supervised learning with contrastive adjusted zooming," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 7, pp. 8882–8896, Sep. 2024.
- [34] K. Zhu, P. Song, and C. Zhao, "Fuzzy state-driven cross-time spatial dependence learning for multivariate time-series anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Mar. 8, 2024, doi: 10.1109/TNNLS.2024.3371109.
- [35] L. Zhao and L. Akoglu, "On using classification datasets to evaluate graph outlier detection: Peculiar observations and new insights," *Big Data*, vol. 11, no. 3, pp. 151–180, Jun. 2023.
- [36] C. Qiu, M. Kloft, S. Mandt, and M. Rudolph, "Raising the bar in graph-level anomaly detection," in *Proc. Thirty-First Int. Joint Conf. Artif. Intell.*, Jul. 2022, pp. 2196–2203.
- [37] A. Protogerou, S. Papadopoulos, A. Drosou, D. Tzovaras, and I. Refanidis, "A graph neural network method for distributed anomaly detection in IoT," *Evolving Syst.*, vol. 12, no. 1, pp. 19–36, Mar. 2021.
- [38] Z. Qin, X. Lu, X. Nie, D. Liu, Y. Yin, and W. Wang, "Coarse-to-fine video instance segmentation with factorized conditional appearance flows," *IEEE/CAA J. Autom. Sinica*, vol. 10, no. 5, pp. 1192–1208, May 2023.

- [39] W. Hu et al., "Spatio-temporal graph convolutional networks via view fusion for trajectory data analytics," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4608–4620, Apr. 2023.
- [40] H. Yuan, J. Bi, and M. Zhou, "Spatiotemporal task scheduling for heterogeneous delay-tolerant applications in distributed green data centers," *IEEE Trans. Autom. Sci. Eng.*, vol. 16, no. 4, pp. 1686–1697, Oct. 2019.
- [41] Y. Wang, T. Xu, X. Niu, C. Tan, E. Chen, and H. Xiong, "STMARL: A spatio-temporal multi-agent reinforcement learning approach for cooperative traffic light control," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 2228–2242, Jun. 2022.
- [42] K. Zhang et al., "Self-supervised learning for time series analysis: Taxonomy, progress, and prospects," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 10, pp. 6775–6794, Oct. 2024.
- [43] M. C. Schiappa, Y. S. Rawat, and M. Shah, "Self-supervised learning for videos: A survey," *ACM Comput. Surv.*, vol. 55, no. 13s, pp. 1–37, Dec. 2023.
- [44] L. Wu, H. Lin, C. Tan, Z. Gao, and S. Z. Li, "Self-supervised learning on graphs: Contrastive, generative, or predictive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 4216–4235, Apr. 2023.
- [45] X. Liu, Y. Liang, C. Huang, Y. Zheng, B. Hooi, and R. Zimmermann, "When do contrastive learning signals help spatio-temporal graph forecasting?" in *Proc. 30th Int. Conf. Adv. Geographic Inf. Syst.*, Nov. 2022, pp. 1–12.
- [46] Q. Zhang, C. Huang, L. Xia, Z. Wang, Z. Li, and S. Yiu, "Automated spatio-temporal graph contrastive learning," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 295–305.
- [47] S. Ranshous, S. Shen, D. Koutra, S. Harenberg, C. Faloutsos, and N. F. Samatova, "Anomaly detection in dynamic networks: A survey," *Wiley Interdiscipl. Rev., Comput. Stat.*, vol. 7, no. 3, pp. 223–247, Mar. 2015.
- [48] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Mar. 2020.
- [49] W. Ren and R. W. Beard, *Distributed Consensus in Multi-Vehicle Cooperative Control*, vol. 27. Cham, Switzerland: Springer, 2008.
- [50] Q. He, Z. Dong, F. Chen, S. Deng, W. Liang, and Y. Yang, "Pyramid: Enabling hierarchical neural networks with edge computing," in *Proc. ACM Web Conf.*, 2022, pp. 1860–1870.
- [51] J. Skarding, B. Gabrys, and K. Musial, "Foundations and modeling of dynamic networks using dynamic graph neural networks: A survey," *IEEE Access*, vol. 9, pp. 79143–79168, 2021.
- [52] S. Wang, J. Cao, and P. S. Yu, "Deep learning for spatio-temporal data mining: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 8, pp. 3681–3700, Aug. 2022.
- [53] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inform. Process. Syst. (NIPS)*, 2017, pp. 5998–6008.
- [54] J.-B. Grill et al., "Bootstrap your own latent: A new approach to self-supervised learning," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, Dec. 2020, pp. 21271–21284.
- [55] D. Grattarola, D. Zamboni, F. M. Bianchi, and C. Alippi, "Understanding pooling in graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 2, pp. 2708–2718, May 2024.
- [56] D. Lee, S. Lee, and H. Yu, "Learnable dynamic temporal pooling for time series classification," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 9, pp. 8288–8296.
- [57] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015. [Online]. Available: <https://openreview.net/forum?id=8gmWwjFyLj> and <https://arxiv.org/pdf/1412.6980>
- [58] J. Sun, J. Zhang, Q. Li, X. Yi, Y. Liang, and Y. Zheng, "Predicting citywide crowd flows in irregular regions using multi-view graph convolutional networks," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 5, pp. 2348–2359, May 2022.
- [59] F. Li, J. Feng, H. Yan, D. Jin, and Y. Li, "Crowd flow prediction for irregular regions with semantic graph attention network," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 5, pp. 1–14, Oct. 2022.
- [60] H. Hua, Y. Li, T. Wang, N. Dong, W. Li, and J. Cao, "Edge computing with artificial intelligence: A machine learning perspective," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, Sep. 2023.
- [61] T. Gong, L. Zhu, F. R. Yu, and T. Tang, "Edge intelligence in intelligent transportation systems: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 9, pp. 8919–8944, Sep. 2023.
- [62] S. Tisue and U. Wilensky, "NetLogo: A simple environment for modeling complexity," in *Proc. Int. Conf. Complex Syst.*, vol. 21, 2004, pp. 16–21.
- [63] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," in *Proc. 14th Annu. Conf. Comput. Graph. Interact. Techn.*, Aug. 1987, pp. 25–34.
- [64] P. Karich and S. Schöder, *Graphhopper, Version 0.4*. Accessed: Mar. 5, 2015. [Online]. Available: <https://graphhopper.com/>
- [65] C. Truong, L. Oudre, and N. Vayatis, "Selective review of offline change point detection methods," *Signal Process.*, vol. 167, Feb. 2020, Art. no. 107299.
- [66] G. J. J. van den Burg and C. K. I. Williams, "An evaluation of change point detection algorithms," 2020, *arXiv:2003.06222*.
- [67] P. Arbelaez, M. Maire, C. Fowlkes, and J. Malik, "Contour detection and hierarchical image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 5, pp. 898–916, May 2010.
- [68] P. Jaccard, "The distribution of the flora in the Alpine zone. 1," *New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912.
- [69] S. Deldari, D. V. Smith, H. Xue, and F. D. Salim, "Time series change point detection with self-supervised contrastive predictive coding," in *Proc. Web Conf.*, Apr. 2021, pp. 3124–3135.
- [70] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," 2018, *arXiv:1803.01271*.
- [71] C. Qiu, T. Pfommer, M. Kloft, S. Mandt, and M. Rudolph, "Neural transformation learning for deep anomaly detection beyond images," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 8703–8714.
- [72] X. Wang, Q. Kang, M. Zhou, L. Pan, and A. Abusorrah, "Multiscale drift detection test to enable fast learning in nonstationary environments," *IEEE Trans. Cybern.*, vol. 51, no. 7, pp. 3483–3495, Jul. 2021.
- [73] C. Gao et al., "S3: Social-network simulation system with large language model-empowered agents," 2023, *arXiv:2307.14984*.
- [74] J. Zhang, Z. Wei, Z. Yan, M. Zhou, and A. Pani, "Online change-point detection in sparse time series with application to online advertising," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 49, no. 6, pp. 1141–1151, Jun. 2019.
- [75] S. Chen and J. Wang, "Heterogeneous interaction modeling with reduced accumulated error for multiagent trajectory prediction," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 8040–8052, Jun. 2024.



Siyuan Chen received the B.S. degree in mathematics and applied mathematics and the Ph.D. degree in computer science and technology from Sun Yat-sen University, Guangzhou, China, in 2018 and 2023, respectively.

He is currently a Lecturer with the School of Computer Science and Cyber Engineering, Guangzhou University, Guangzhou. His research interests include graph learning, crowd intelligence, spatio-temporal data mining, and social network mining.



Xin Du received the B.S. degree in computer science and technology from Jiangnan University, Wuxi, China, in 2017, and the Ph.D. degree in computer science and technology from Sun Yat-sen University, Guangzhou, China, in 2023.

He is currently a Lecturer with the Civil Aviation Electronic Information Engineering College, Guangzhou Civil Aviation College, Guangzhou. His research interests include deep learning and deep reinforcement learning.



Jiahai Wang (Senior Member, IEEE) received the Ph.D. degree in computer science from the University of Toyama, Toyama, Japan, in 2005.

In 2005, he joined Sun Yat-sen University, Guangzhou, China, where he is currently a Full Professor with the School of Computer Science and Engineering. He is currently leading the Intelligent Optimization and Learning Laboratory, Sun Yat-sen University. He has published a series of papers in leading conferences and top journals including AAAI, ACL, and IEEE TRANSACTIONS. His main

research interests include computational intelligence (deep neural networks and metaheuristics) and its applications.

Prof. Wang is a Distinguished Member of CCF, China.